

Getting started

From a fresh Windows computer to your first scan. Type each command into **PowerShell**: press Start, type PowerShell, press Enter. Copy one block at a time.

1. Install the tools (once)

SecureGate needs four free tools. **winget**, the app installer built into Windows, installs them:

```
winget install --id Git.Git -e
winget install --id Python.Python.3.12 -e
winget install --id Gitleaks.Gitleaks -e
winget install --id ezwinports.make -e
```

- **Git** keeps the history of a project.
- **Python** runs SecureGate.
- **Gitleaks** is the scanner that spots key-shaped text.
- **make** runs short commands such as `make setup`.

Close PowerShell and open it again, so that it finds the new tools. Then check them:

```
git --version
py -3.12 --version
gitleaks version
make --version
```

Each command prints a version number. If one says "not recognized", close PowerShell and open it once more.

2. Get SecureGate

```
cd $HOME\Documents
git clone <repository-address> SecureGate
cd SecureGate
```

Replace `<repository-address>` with the address your team gave you. `cd` means "change directory": it moves you into a folder.

3. Set it up (once)

```
make setup
```

This creates a private Python space called `.venv` inside the folder and installs SecureGate into it. It takes a minute or two.

4. Your first scan: the demo repo

```
make demo
make scan-demo
```

`make demo` builds a small fake payments app full of made-up secrets, in a folder called `securegate-demo` next to the SecureGate folder. `make scan-demo` scans it. You will see a table like this:

```
DECISION  RULE                FILE:LINE                VALUE
block     github-pat          .env:2                   ghp_****pVJI
block     aws-access-token    config/settings.py:11    AKIA****NOBM
block     private-key         deploy/server.key:1      ----****----
...
11 findings: 6 block, 4 warn, 1 ignore -> BLOCKED (exit code 1). Details: findings-demo.json
make: *** [Makefile:43: scan-demo] Error 1
```

The last two lines are expected. The demo is full of secrets, so SecureGate blocks it (exit code 1), and `make` reports that as "Error 1". Every value is masked: SecureGate never shows a whole secret.

5. Scan a real project

Stay in the SecureGate folder, because it holds the rules, and point SecureGate at your project:

```
.venv\Scripts\securegate scan C:\path\to\your\project --mode repo
```

`--mode repo` reads the project's whole history. Other modes:

- `dir`: only the files as they are now.
- `staged`: only the changes you are about to commit.
- `range`: only some commits, for example `--mode range --range main..my-branch`.

The first scan creates a `.securegate` folder holding a private key for fingerprints. Keep it. Git ignores it automatically.

6. See the results in your browser

```
make ui
```

This opens the dashboard on the demo scan at `http://127.0.0.1:5000`, a page only your own computer can open. Select a finding to see how to fix it. Press Ctrl+C in PowerShell to stop the dashboard. For your own scan, use `.venv\Scripts\securegate ui --report findings.json --open`. More in Dashboard (ui.pdf).

7. Check every commit before it is made (once)

```
make hooks
```

This turns on the **laptop gate**: from now on, every `git commit` in this folder first scans the changes you are about to commit. If it finds a secret to block, the commit stops and shows the file and line, the masked value, why it was blocked and how to fix it. The first run of `make hooks` needs the internet; after that it works offline. More in [The two gates \(merge-gate.pdf\)](#).

On macOS or Linux

Install the same tools, then follow steps 2 to 7 in a terminal.

```
# macOS, with Homebrew (https://brew.sh):
brew install git python@3.12 gitleaks
xcode-select --install          # provides make

# Ubuntu or Debian Linux:
sudo apt install git make python3.12 python3.12-venv
# Gitleaks for Linux: https://github.com/gitleaks/gitleaks/releases
```

In steps 5 and 6, use `.venv/bin/securegate` instead of `.venv\Scripts\securegate`.